

Capítulo 8 — GUÍA DE LECTURA

Capítulo 8 — GUÍA DE LECTURA.....	1
Arquitectura: Feature-Sliced Design, estado y el TPI	2
Dónde va cada cosa, y el cierre del módulo, explicados en criollo	2
Antes de empezar: cómo usar esta guía.....	2
8.1 — De qué se trata esta clase	3
Qué dice	3
En criollo.....	3
8.2 — Por qué organizar por tipo de archivo se cae a pedazos	4
Qué dice	4
En criollo.....	4
La respuesta no es nueva, y no nació en el frontend	4
8.3 — Las capas del TPI.....	5
8.3.1 — La regla de dependencias, y sus dos prohibiciones	5
Qué dice	5
En criollo.....	6
8.3.2 — Las nueve piezas y qué le toca a cada una	7
Dos filas son decisiones, no descripciones.....	7
8.3.3 — La pieza que no entra en ninguna capa	7
8.4 — Cada dato tiene un dueño: estado del cliente y estado del servidor	8
8.4.1 — La distinción que más ordena un frontend	8
Qué dice	8
En criollo.....	8
8.4.2 — Los seis stores, y qué sobrevive a cerrar el navegador.....	9
Las tres decisiones que no son obvias	10
8.4.3 — La excepción declarada, y por qué el campo se llama <code>precio_ref</code>	10
8.5 — El puente entre los eventos y el resto del sistema.....	10
8.6 — El arranque en siete pasos y la regla RN-F06.....	11
Por qué «nunca la vista de login» no es una preferencia estética	12
8.7 — Las guardas de ruta: la regla RN-F04 como decisión de arquitectura	13
8.8 — El checkout y la regla RN-F07: la deuda que el Capítulo 1 dejó abierta	13
8.9 — Las once reglas, vistas como un sistema	15
8.10 — Trabajar con un agente de IA sobre esta base	16
¿Y qué significa eso sentado frente a un agente?	16
8.11 — Herramientas de diagnóstico: auditar la arquitectura desde afuera	17
8.12 — Seguridad y evolución: qué de todo esto va a seguir sirviendo.....	18
8.13 — Verificación: el checklist honesto.....	19
8.14 — Los doce errores frecuentes	19

8.15 — Las actividades, y qué busca cada una	20
1. Mapa del proyecto	20
2. Auditoría de dependencias.....	20
3. La distinción, aplicada	20
4. El arranque completo.....	20
5. El ciclo de la clave	20
6. Exploración: revisión de código generado	21
7. Exploración: la arquitectura auditada desde afuera.....	21
8.16 — Síntesis: las doce frases	21
8.17 — Qué leer, y en qué orden	21
Si lees una sola cosa.....	22
Si lees tres	22
Las secciones del TPI que conviene tener abiertas	22
Y como cierre del módulo, tres lecturas que exceden lo técnico.....	22
Cierre: las ocho cosas que hay que recordar	22

Arquitectura: Feature-Sliced Design, estado y el TPI

Dónde va cada cosa, y el cierre del módulo, explicados en criollo

Traducción didáctica del texto académico, sección por sección. Mismos conceptos, otro idioma.

Antes de empezar: cómo usar esta guía

Este documento no reemplaza al capítulo original: lo traduce, y **no se pierde ni un concepto**. Cada sección tiene tres partes: **Qué dice** —la idea del original—, **En criollo** —la explicación larga con su analogía— y **Para el pizarrón**, la frase que te tenés que llevar.

Una advertencia que vale sólo para éste: **es el último capítulo**. Los otros siete te dieron piezas; éste **las ordena**.

LA IDEA MADRE DE TODO EL CAPÍTULO

Una arquitectura no es un dibujo lindo: es un acuerdo sobre dónde va cada cosa y qué le puede pedir cada pieza a las demás.

Y hay una segunda mitad, que decide si el acuerdo sirve: **un acuerdo que depende de que todos se acuerden, se rompe.**

Por eso el capítulo no te pide que seas prolijo: te muestra estructuras donde el desorden **no entra** — un grafo que delata la importación prohibida, una secuencia donde montar antes de tiempo no existe, y un tipo que no te deja olvidarte del tercer estado.

8.1 — De qué se trata esta clase

Qué dice

Los siete capítulos anteriores dejaron todas las piezas construidas; éste las ordena. La pregunta que lo abre no es técnica sino de método: con setenta endpoints, veintitrés entidades y cinco personas en paralelo, **¿dónde va cada cosa, y qué le puede pedir cada pieza a las demás?** Un sistema sin respuesta a eso no falla de golpe: se vuelve cada vez más lento de modificar, hasta que cada cambio requiere entender el conjunto entero.

En criollo

Fijate en la forma de ese fracaso, que es lo que lo vuelve peligroso. **Un sistema mal organizado no explota:** la tarea que en marzo llevaba dos horas, en septiembre lleva dos días, y nadie sabe por qué.

La razón es que cada cambio te obliga a cargar más contexto: si para tocar el carrito abrí cinco carpetas y lees archivos de otras tres funcionalidades, **el costo de cada cambio incluye entender el conjunto entero** — que crece todos los días.

Cuatro de las once reglas obligatorias se fundan acá, y con ellas el módulo cierra las once:

Regla	Qué ordena	Qué venía arrastrándose
RN-F03	Separa el estado del cliente del estado del servidor	La distinción que más ordena un frontend
RN-F06	Ordena la secuencia de arranque	El parpadeo del login, que el Capítulo 2 anticipó
RN-F07	Resuelve la idempotencia del checkout	El Capítulo 1 lo dejó planteado: POST no es idempotente, y reintentar duplica el pedido
RN-F04	Fija que las guardas son usabilidad, no seguridad	Anticipada dos veces, en 1.9.2 y al hablar de validación

Y hay un cierre que le debe al módulo: el TPI declara en su sección 2.4 que la entrega **asume el trabajo con un agente de IA**, bajo este criterio:

La herramienta no sustituye la comprensión: el documento está escrito para que **ninguna de sus decisiones pueda copiarse sin entenderse**, porque cada una viene acompañada de su porqué y de su alternativa descartada.

Ése es, palabra por palabra, el método de los ocho capítulos, y la sección 8.10 lo vuelve operativo: qué pedirle a un agente, qué revisarle, y qué rompe **por defecto** si nadie se lo impide.

PARA EL PIZARRÓN

Al terminar tenés que poder **agarrar cualquier archivo del frontend del TPI y decir en qué capa vive**, justificar qué puede importar, y explicar las once reglas como un sistema y no como una lista suelta.

La prueba casera: elegí un archivo al azar y preguntate *¿de quién puede depender, y quién puede depender de él?*

8.2 — Por qué organizar por tipo de archivo se cae a pedazos

Qué dice

La forma más difundida de organizar un frontend agrupa los archivos por **qué son**: componentes con componentes, servicios con servicios. Funciona hasta unos cuantos miles de líneas y después produce cuatro problemas, de cada uno de los cuales sale una decisión de Feature-Sliced Design.

En criollo

Un proyecto organizado así se ve de esta manera:

Carpeta	Contiene
src/components/	Boton.ts · TarjetaProducto.ts · FilaPedido.ts · Grafico.ts ...
src/services/	productos.ts · pedidos.ts · auth.ts · estadisticas.ts ...
src/utils/	formato.ts · validacion.ts · fechas.ts ...
src/types/	producto.ts · pedido.ts · usuario.ts ...
src/store/	carrito.ts · sesion.ts · ui.ts ...

Cinco carpetas, y fíjate: **ninguna dice de qué se trata el sistema**. No es un detalle estético: son cuatro problemas concretos, y de cada uno sale una decisión de diseño.

El problema	Cómo se siente	Qué decisión sale de acá
1. Un cambio se reparte por todo el proyecto	Para tocar el carrito abrí las cinco carpetas, y ninguna te dice cuáles archivos son suyos	Que todo lo de una funcionalidad viva junto
2. La estructura no dice qué hace el sistema	No sabés si es una tienda o un juego: dice con qué está hecho, no de qué se trata . Martin llamó a lo contrario <i>arquitectura que grita</i>	Organizar por dominio, no por tipo
3. Todo puede importar todo	Nada impide que una utilidad importe un store, y aparecen las importaciones circulares del Capítulo 3	Declarar capas con una dirección obligatoria
4. Borrar es imposible	Hay que hallar los archivos entre los ajenos y ver si alguien más los usa; como nunca estás seguro, no se borra — lo del Capítulo 2 con las hojas de estilo	Que cada funcionalidad sea una unidad

La respuesta no es nueva, y no nació en el frontend

Esto no lo inventó nadie en 2021. La **arquitectura hexagonal** de Alistair Cockburn (2005) y la **arquitectura limpia** de Robert C. Martin (2012) ya proponían lo esencial: **organizar por dominio, y declarar una dirección para las dependencias**. Las dos nacieron en el backend, y son lo que el TPI aplica en su sección 2.1.

Feature-Sliced Design las adapta al frontend, formalizada por la comunidad alrededor de 2021, y sus decisiones son sólo dos. **Primera: organizar por funcionalidad** — todo lo del carrito vive junto, sea

componente, servicio o tipo. **Segunda: declarar capas con una dirección obligatoria** — cada capa importa de las de abajo y **nunca** de las de arriba. Todo lo que sigue son consecuencias de esas dos.

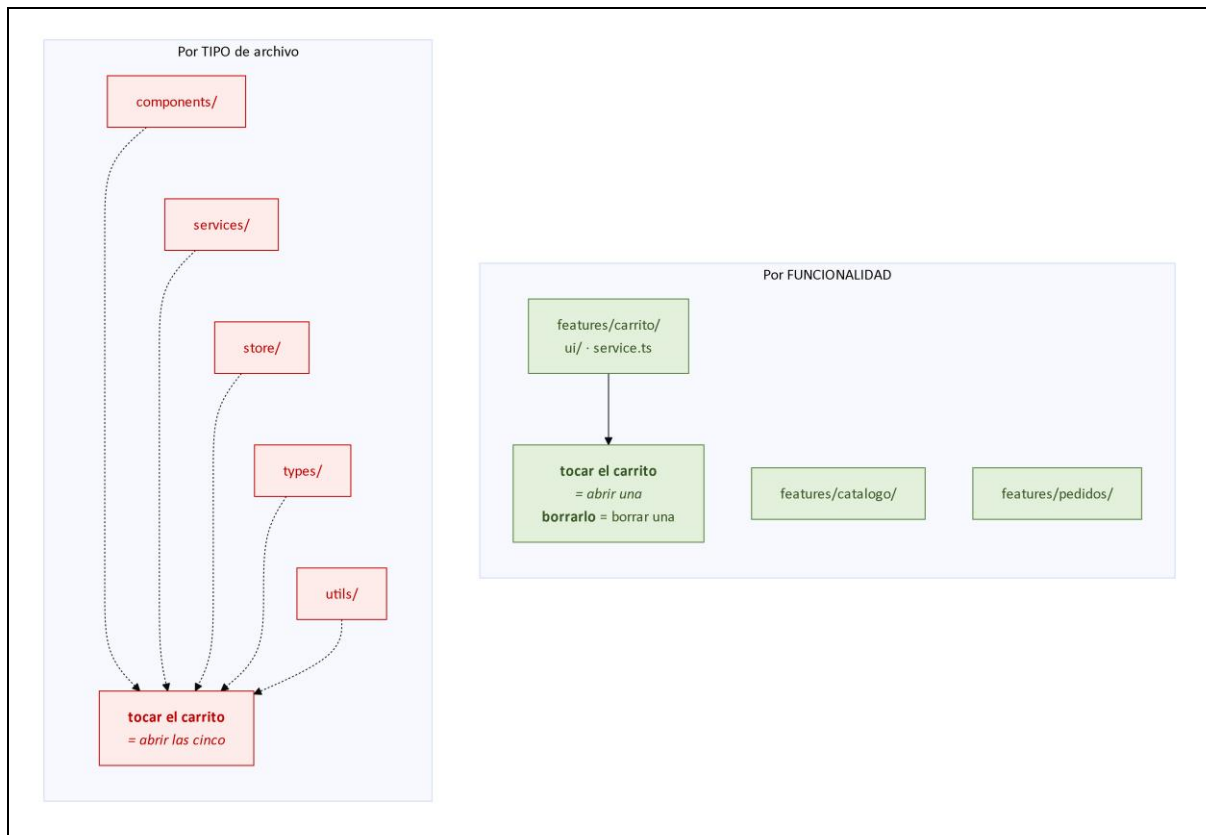


Figura 8.1. Por tipo frente a por funcionalidad

PARA ENTENDER: el problema que no se ve es el que más cuesta

El cuarto es el más caro y el que menos se nota. **Un proyecto donde no se puede borrar sólo puede crecer**, y no con cosas útiles: crece con restos que nadie usa, que nadie se anima a tocar, y que igual hay que leer cada vez que buscás algo.

Cuando todo lo de una funcionalidad vive en una carpeta y las dependencias tienen una sola dirección, borrar es trivial: **borrás la carpeta y el compilador te dice quién la extrañaba**. Si no protesta nadie, no la necesitaba nadie.

Ésa es la prueba de fuego, y te la podés hacer hoy sobre cualquier proyecto tuyo: *¿puedo borrar esta funcionalidad y saber en treinta segundos qué se rompió?*

8.3 — Las capas del TPI

8.3.1 — La regla de dependencias, y sus dos prohibiciones

Qué dice

El TPI declara la regla en su sección 2.4, en una sola frase con dos prohibiciones:

Los imports fluyen de arriba hacia abajo: **Arranque → Router → Vistas → Features (ui y service) → Stores, Cliente API y Cliente de eventos → Types**. Ninguna capa importa la capa superior y ninguna feature importa de otra feature en horizontal.

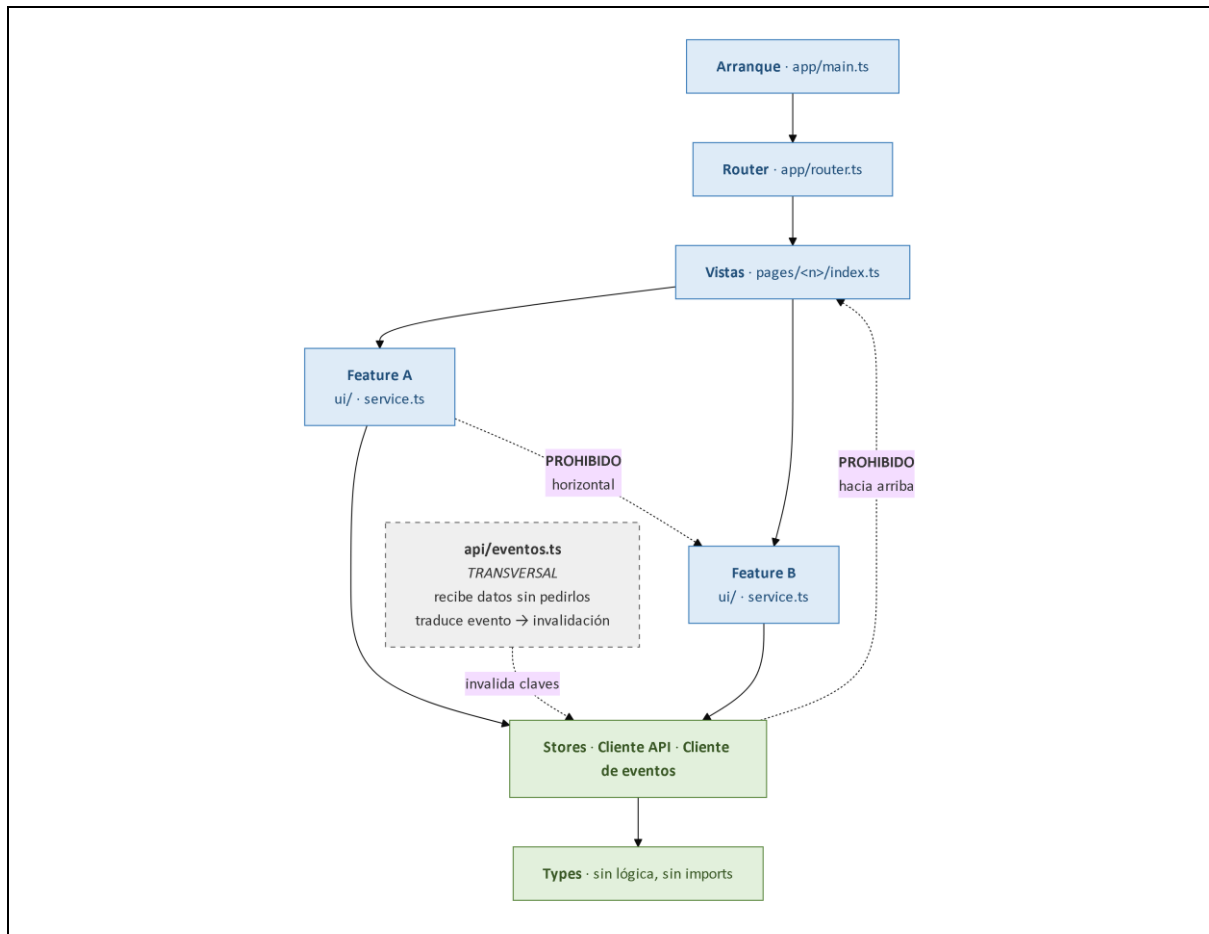


Figura 8.2. Las capas y la dirección de las dependencias

En criollo

Dos prohibiciones que no resuelven lo mismo:

La prohibición	Qué garantiza	Qué se rompe si la violás
No importar hacia arriba	Que las capas de abajo sean independientes : un store no sabe qué vista lo usa	Deja de probarse solo y de reutilizarse: arrastra medio proyecto atrás de sí
No importar en horizontal	Que cada carpeta siga siendo una unidad — la promesa de 8.2	Si pedidos importa de catálogo, borrar catálogo rompe pedidos: volviste al problema cuatro

¿Y si dos funcionalidades necesitan lo mismo? No se importan entre ellas: **eso baja a una capa inferior**, donde ambas pueden verlo sin conocerse. Es la única maniobra permitida.

PARA EL PIZARRÓN: las capas son calles de mano única

No están así por manía del orden, sino **para que dos autos no se encuentren de frente**. La flecha hacia arriba es ir contramano; la horizontal es cruzar por la vereda del vecino — llegás, pero el día que esa casa se demuele te quedaste sin paso. Hay una sola forma legal de compartir algo: **bajarlo de piso**.

8.3.2 — Las nueve piezas y qué le toca a cada una

La sección 2.4 del TPI declara cada archivo y su responsabilidad. Ésta es la tabla que conviene tener a mano todo el cuatrimestre:

Pieza	Archivo	Responsabilidad
Arranque	app/main.ts	Rehidrata la sesión antes de resolver la primera ruta
Router	app/router.ts	Mapea la ruta a una vista; invoca <code>destroy()</code> y <code>mount()</code>
Vista	pages/<n>/index.ts	Expone <code>mount(container, params)</code> y <code>destroy()</code>
Componente	features/<f>/ui/<c>.ts	Elemento personalizado con su ciclo de vida. Recibe datos
Servicio	features/<f>/service.ts	Encapsula el observador de consultas
Store	store/<n>Store.ts	Estado del cliente
Cliente API	api/<recurso>.ts	Funciones tipadas. Convierte los importes
Cliente de eventos	api/eventos.ts	Abre y cierra el canal; traduce eventos en invalidaciones
Tipos	types/<dominio>.ts	Interfaces espejo de los esquemas. Sin lógica

Mírala con los ocho capítulos encima, porque **no hay una pieza nueva**: el componente es el Capítulo 7; el cliente API, el 5 tipado con el 6; el de eventos, la 5.9; los tipos, la 6.9; la vista y el router, la History API; el store, `zustand/vanilla` con su baja de RN-F01. **Este capítulo les asigna lugar.**

Dos filas son decisiones, no descripciones

El componente recibe datos. No los busca. Uno que llama a la API queda atado a una ruta y deja de ser probable sin servidor, como estableció el Capítulo 7: los datos entran por propiedad, y el que los busca es su servicio.

El cliente API convierte los importes. Ése es el lugar único donde ocurre la conversión de RN-F08. En la vista **habría tantas conversiones como vistas**, y para mostrar mal un precio bastaría con que a una le faltara. Un lugar se revisa; doce, no.

8.3.3 — La pieza que no entra en ninguna capa

Hay una excepción, y lo interesante es que el TPI **la declara** en vez de dejarla implícita:

La pieza `api/eventos.ts` es **transversal**: es el segundo lugar del frontend que conoce una ruta del backend, y **el primero que recibe datos sin haberlos pedido**.

Esa segunda mitad es la que importa. El resto del sistema funciona por petición: alguien pide, algo responde. **El canal de eventos rompe ese modelo**: llegan datos que nadie pidió, en cualquier momento, sin que haya una vista esperándolos. Por eso no encaja en ninguna capa, y por eso su responsabilidad está acotada con precisión: **traduce cada evento en una invalidación de clave, y nada más**. No actualiza vistas, no escribe en stores, no decide nada. Es RN-F09 expresada como arquitectura.

NOTA: una regla con excepciones declaradas sigue siendo una regla

La excepción **está declarada, nombrada y acotada**: dice cuál pieza es, por qué es distinta y qué le está permitido hacer.

Compará con la alternativa habitual: una regla linda en el README y tres lugares que la violan porque «era más práctico». Como no están documentadas, el próximo que llega no sabe si son legítimas o descuidos — y ante la duda, **agrega la suya**.

Una regla con excepciones declaradas es una regla; con excepciones silenciosas es una sugerencia. Lo mismo pasa en RN-F03 y en RN-F08.

8.4 — Cada dato tiene un dueño: estado del cliente y estado del servidor

8.4.1 — La distinción que más ordena un frontend

Qué dice

El estado del servidor vive en la base de datos y el cliente tiene una copia temporal; el del cliente sólo existe en el navegador. Tratarlos igual produce la mayoría de los bugs de estado, y de ahí sale RN-F03.

En criollo

La distinción central del capítulo se hace con una pregunta: **¿quién es el dueño de este dato?**

	Estado del servidor	Estado del cliente
Dueño	La base de datos; vos tenés una copia temporal	El navegador, y nadie más
Ejemplos	Productos, pedidos, stock, el usuario y su rol	El modal abierto, el filtro, el carrito sin confirmar
¿Queda vieja?	Sí : otro pudo cambiarla hace un segundo	No : no hay contra qué compararla
Dónde vive	El QueryClient, la caché del Capítulo 5	Un store de <code>zustand/vanilla</code>
Qué problema trae	Cuándo recargarlo, cómo invalidarlo, qué hacer si otro lo cambió	Ninguno: nadie más lo toca

Mirá la última fila: si metés el estado del servidor en un store, **te quedás con los problemas de la izquierda y sin lo que los resuelve**.

RN-F03. El estado del servidor vive únicamente en el QueryClient y el del cliente únicamente en los stores; **la única excepción declarada son los campos de exhibición de `CartItem`**. Garante: revisión — sin garante automatizado, declarado como tal.

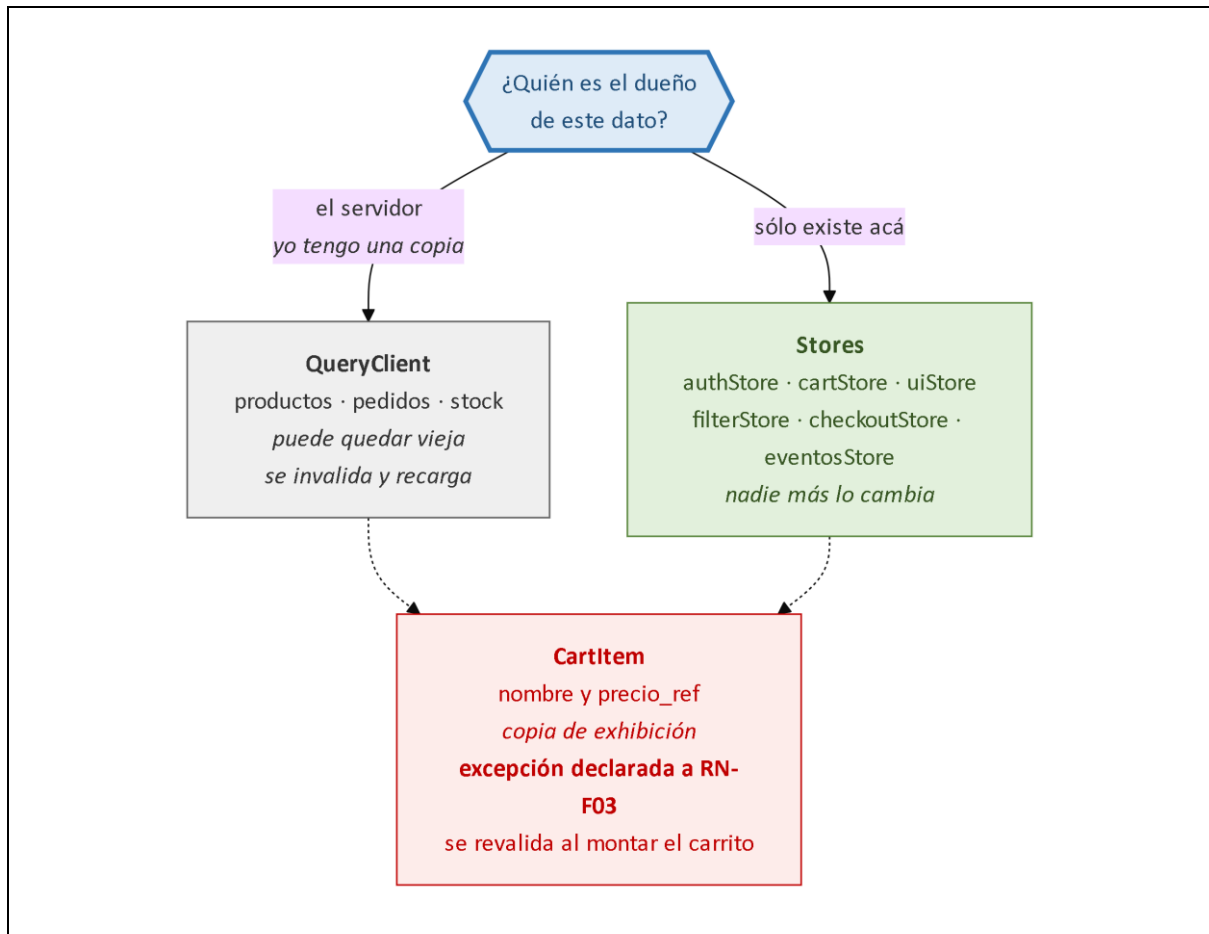


Figura 8.3. Estado del cliente frente a estado del servidor

Ojo con la última frase, la más honesta de las once: **declara que no tiene garante automatizado** en lugar de fingir que sí. No es debilidad, es información: dice dónde poner ojos humanos porque ningún test va a avisar.

OJO ACÁ: el error que más bugs de estado produce

- Si el dueño es el servidor y vos tenés una copia → **QueryClient**, y hay que tener una estrategia para cuando quede vieja.
- Si el dato **sólo existe acá** → **store**. Nadie más lo puede cambiar.

El error clásico es meter la lista de pedidos en un store «para tenerla a mano». Y ahí empezás: ¿cuándo la recargo? ¿qué hago si llegó un evento? **Estás reimplementando a mano una caché de consultas, mal y sin darte cuenta.**

Y cuidado: si le pedís a un agente que te maneje «el estado de la aplicación», **te va a meter todo en un solo store. Vos tenés que hacer el corte.**

8.4.2 — Los seis stores, y qué sobrevive a cerrar el navegador

La sección 13.1 del TPI los declara y —lo más importante— **qué persiste cada uno**:

Store	Gestiona	Persiste
authStore	Token, usuario, autenticado	Sólo el token
cartStore	El carrito	Los ítems completos

Store	Gestiona	Persiste
uiStore	Modales, panel, notificaciones, tema	Nada
filterStore	Filtros del catálogo y del panel	Nada
checkoutStore	Paso, dirección, pago, clave de idempotencia	Paso, forma, modalidad y clave
eventosStore	La conexión y el último id recibido	Sólo el último id

Los seis se componen igual, y esa uniformidad es deliberada: **el que entiende uno entiende los seis**. Con Zustand/vanilla, qué sobrevive se decide en una línea —el `partialize` de `persist`, que en `authStore` deja pasar el token y descarta el usuario—, **y una línea se revisa fácil**.

Las tres decisiones que no son obvias

authStore persiste el token y no el usuario, porque los datos del usuario pueden haber cambiado —el rol, por ejemplo— y se vuelven a pedir en cada arranque, como muestra 8.6. Persistirlo sería **estado del servidor guardado en un store**: lo que RN-F03 prohíbe, y a quien le cambiaron el rol seguiría viendo el panel.

uiStore y filterStore no persisten nada, porque un modal abierto o un filtro elegido **no deberían sobrevivir a cerrar el navegador**: volvés tres días después y no entendés por qué faltan productos.

eventosStore persiste el último id, que es lo que permite que **una recarga no pierda el hueco de eventos** —la sección 5.9.3 llevada al que aprieta F5—. Sin eso, cada recarga se conecta sin punto de referencia y lo publicado mientras tanto se pierde.

8.4.3 — La excepción declarada, y por qué el campo se llama `precio_ref`

`CartItem` guarda nombre y `precio_ref` —una copia de exhibición de datos del servidor—, y es la excepción declarada a RN-F03. La razón se prueba en diez segundos: **el carrito tiene que poder mostrarse sin pedir nada**. Si sólo guardara identificadores, abrirlo sin conexión mostraría un carrito vacío, y el usuario pensaría que perdió lo que había elegido.

Pero una copia puede quedar vieja: **el precio pudo cambiar**. Por eso la excepción viene con su contrapartida —al montar la vista del carrito se revalida— y por eso el campo **se llama `precio_ref` y no `precio`**: es una referencia de exhibición, no el precio de la transacción. El que vale lo calcula el servidor al confirmar, que es lo que sostiene RN-F08.

NOTA: el nombre de una variable también es documentación

Dentro de seis meses alguien —tal vez vos— va a escribir el checkout apurado y va a ver ese campo listo para multiplicar por la cantidad. Si se llamara `precio`, lo haría sin pensarlo; **como se llama `precio_ref`, la pregunta aparece sola: referencia, ¿de qué?**

8.5 — El puente entre los eventos y el resto del sistema

El servicio de cada funcionalidad encapsula un **observador de consultas**: una pieza que pide un dato, lo cachea bajo una clave, avisa cuando cambia y lo vuelve a pedir —lo del Capítulo 5, con `@tanstack/query-core`—. Para la arquitectura importa **qué te ahorra de escribir a mano**: caché por clave, deduplicación de peticiones simultáneas, estados de carga y error, recarga en segundo plano, y el **intervalo de respaldo que RN-F11 exige** —el temporizador que mantiene viva la pantalla si el canal se cayó—. Cinco cosas que, si no las tenés, terminás escribiendo peor y esparcidas por las vistas.

Y acá se cierra el arco del Capítulo 5. **RN-F09** dice que un evento invalida en lugar de escribir, y recién ahora se ve por qué es una decisión de arquitectura: **la invalidación es el único punto de contacto entre el canal y el resto del sistema**. Mirá lo poco que hace la pieza transversal:

```
// api/eventos.ts — la pieza transversal, y todo lo que hace
canal.addEventListener("pedido_actualizado", (evento) => {
  const { pedido_id } = JSON.parse(evento.data);
  queryClient.invalidateQueries({ queryKey: ["pedidos", pedido_id] });
});
```

Tres líneas, y **ninguna toca una vista**. El evento dice *qué mirar de nuevo*; quién lo mira es problema de las capas de arriba, que ni saben que existe un canal. Esa indiferencia prueba que la arquitectura está bien puesta: **si el canal se reemplaza por otra cosa, ninguna vista se entera**.

PARA ENTENDER: la única medida honesta de una arquitectura

Elegí una pieza y preguntate: ¿cuántos archivos tendría que tocar para reemplazarla?

Con el canal bien puesto, la respuesta es **uno**: `api/eventos.ts`. Mal puesto —cada vista suscribiéndose por su cuenta— la respuesta es *«todas las vistas que muestran datos vivos»*, y ahí ya no reemplazás nada: **convivís con lo que hay**.

Hacé esa cuenta con la caché, el cliente HTTP y el store del carrito de tu TPI. **El número que te dé es la medida real de tu arquitectura**, más honesta que cualquier diagrama.

8.6 — El arranque en siete pasos y la regla RN-F06

La sección 13.3 del TPI describe la secuencia de arranque, y no es caprichosa: **cada paso resuelve un problema concreto**, y el orden es la mitad del diseño.

1. **Lee el estado persistido**. Sin token, marca la rehidratación como terminada y le cede el control al enrutador sin abrir ningún canal.
2. **Con token, muestra el estado de carga** —nunca la vista de login— y pide los datos del usuario.
3. **Con respuesta exitosa**, completa el usuario y termina la rehidratación.
4. **Con 401**, limpia la sesión, termina la rehidratación como anónima y no abre el canal.
5. **Recién con la rehidratación terminada**, el enrutador resuelve la ruta y monta la vista sobre una sesión ya poblada.
6. **Precarga en paralelo** las claves de catálogo y el árbol de categorías, que quedan disponibles para todos los formularios de la sesión.
7. **Si hay sesión, abre el único canal**, enviando el último identificador persistido. No elige canal: el servidor los resuelve del actor.

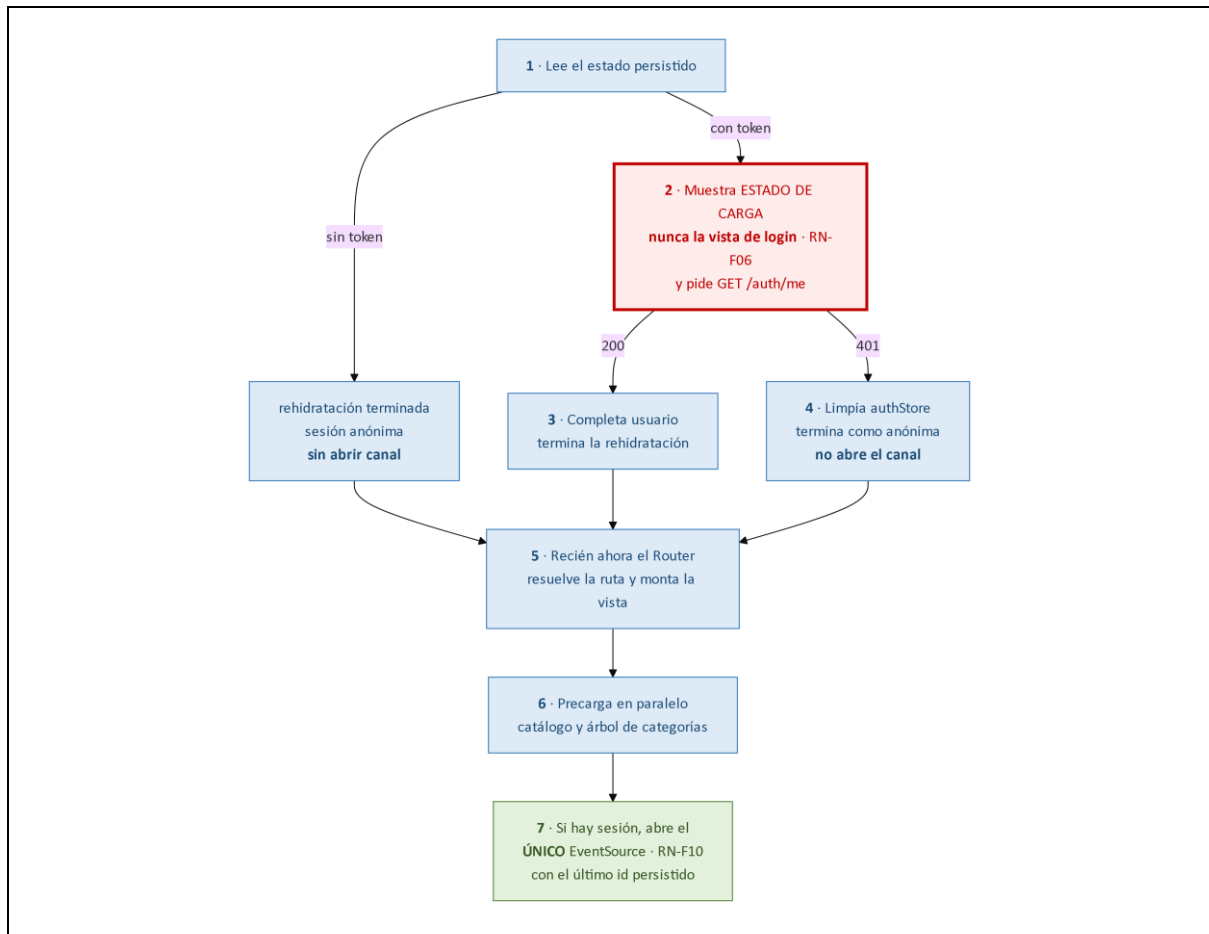


Figura 8.4. Los siete pasos del arranque

Fijate al pasar en el paso 6: **en paralelo**, el criterio del Capítulo 5 sobre los `await` encadenados. Y en el 7: **una sola conexión**, que es RN-F10. Pero el paso que se convirtió en regla es el 2:

RN-F06. Ninguna vista se monta antes de que termine la rehidratación de la sesión; mientras está en curso, el arranque muestra un estado de carga **y nunca la vista de login**. Garante: revisión.

Por qué «nunca la vista de login» no es una preferencia estética

Sin esa regla pasa lo siguiente, y seguro lo viste sin saber qué lo causaba. Hay un token guardado, pero verificarlo tarda trescientos milisegundos: en ese lapso el estado dice «no autenticado» — todavía no se sabe —, el enrutador aplica su guarda y **manda al login a alguien que tiene la sesión abierta**, y al rato la verificación vuelve bien y lo saca.

Resultado: **un parpadeo del login en cada recarga**, de los defectos que más desconfianza generan. Su causa es una: **confundir «todavía no sé» con «no»**.

PARA ENTENDER: el tercer estado que casi todos se olvidan

Este principio excede al TPI: **«todavía no sé» es un estado, y hay que representarlo**.

La mayoría de los frontends tienen dos: autenticado y no autenticado. Ahí nace el parpadeo, porque durante la verificación **estás en un tercer estado que tu código no contempla**, y cae en el que esté por defecto. ¿Te acordás de la unión discriminada del Capítulo 6? Es exactamente esto:

```

type Sesion =
  | { estado: "rehidratando" } // ← el que casi todos se olvidan
  | { estado: "anonima" }
  
```

```
| { estado: "autenticada"; usuario: Usuario };
```

Con ese tipo, el compilador **no te deja** olvidarte del tercer caso: te lo exige en cada switch. El bug **se vuelve imposible de escribir**, que es mucho mejor que acordarse de evitarlo.

8.7 — Las guardas de ruta: la regla RN-F04 como decisión de arquitectura

El enrutador aplica guardas antes de montar una vista: si la ruta exige un rol y el usuario no lo tiene, no la monta. Lo interesante es qué dice el TPI que eso significa **exactamente**:

RN-F04. Las guardas de ruta son **usabilidad, no seguridad**: el backend revalida siempre el rol y la propiedad del recurso. Garante: **TST-06, TST-07 y TST-27, que ejercitan el backend sin pasar por la interfaz.**

Fijate en el garante, lo más elocuente de la regla. **Los tres tests no usan el frontend en absoluto**: le pegan directamente al backend, que es lo que haría un atacante. No hay guarda ni botón oculto en el medio: hay una petición cruda contra un endpoint.

Eso convierte una advertencia en una verificación: no dice «acordate de que el cliente no protege» —frase que nadie puede comprobar—, **demuestra que el servidor protege solo**. Lo que las guardas sí aportan es no mostrar botones que van a fallar ni cargar vistas que van a devolver 403. Es valioso, **y es otra cosa**.

OJO ACÁ: ocultar no es proteger

Una guarda de ruta es **el cartel de «Privado» en una puerta**: la gente honesta no entra, pero el cartel no es una cerradura y el que quiere entrar no lee carteles.

La cerradura está en el backend, siempre. Ocultar el botón de eliminar es prolijidad; impedir que el DELETE /productos/12 funcione es seguridad.

Prueba de un minuto sobre tu TPI: abrí la consola, escribí a mano la petición que el botón oculto habría hecho y mandala. **Si algo se borra, la cerradura no existía.**

8.8 — El checkout y la regla RN-F07: la deuda que el Capítulo 1 dejó abierta

Acá se salda una cuenta abierta en la primera clase. El Capítulo 1 estableció, leyendo la RFC 9110, que **POST no es idempotente**: repetir la petición produce dos efectos, y de esa propiedad de la norma —no de un capricho del enunciado— sale la clave. Ésta es la regla completa:

RN-F07. El checkout genera una clave de idempotencia con `crypto.randomUUID()` **al entrar al último paso**, la persiste en `checkoutStore` y la envía en el encabezado `Idempotency-Key`; **la clave se descarta recién en el onSuccess de la mutación**. Garante: TST-23 y TST-38 del lado del servidor.

Parece tener tres detalles decorativos, y no: **cada uno responde a un caso de fallo distinto**.

El detalle	Contra qué protege	Qué pasa si se hace mal
Se genera al entrar al último paso , no al confirmar	El doble clic	Cada clic daría una clave nueva, y dos clics apurados crearían dos pedidos
Se persiste en el store y sobrevive a una recarga	El corte de red: confirmás, se corta, recargás	Al reintentar mandás una clave distinta : el servidor crea un segundo pedido

El detalle	Contra qué protege	Qué pasa si se hace mal
Se descarta al confirmarse el éxito, no al emitir	La respuesta lenta: mientras no llegue, la clave debe seguir	El usuario reintenta, ya no hay clave, se genera otra, y dos pedidos

En el TPI eso es `crypto.randomUUID()` al entrar al paso, el middleware persist de `zustand/vanilla` guardándola, Axios mandándola en el encabezado y el `onSuccess` de `@tanstack/query-core` limpiándola.

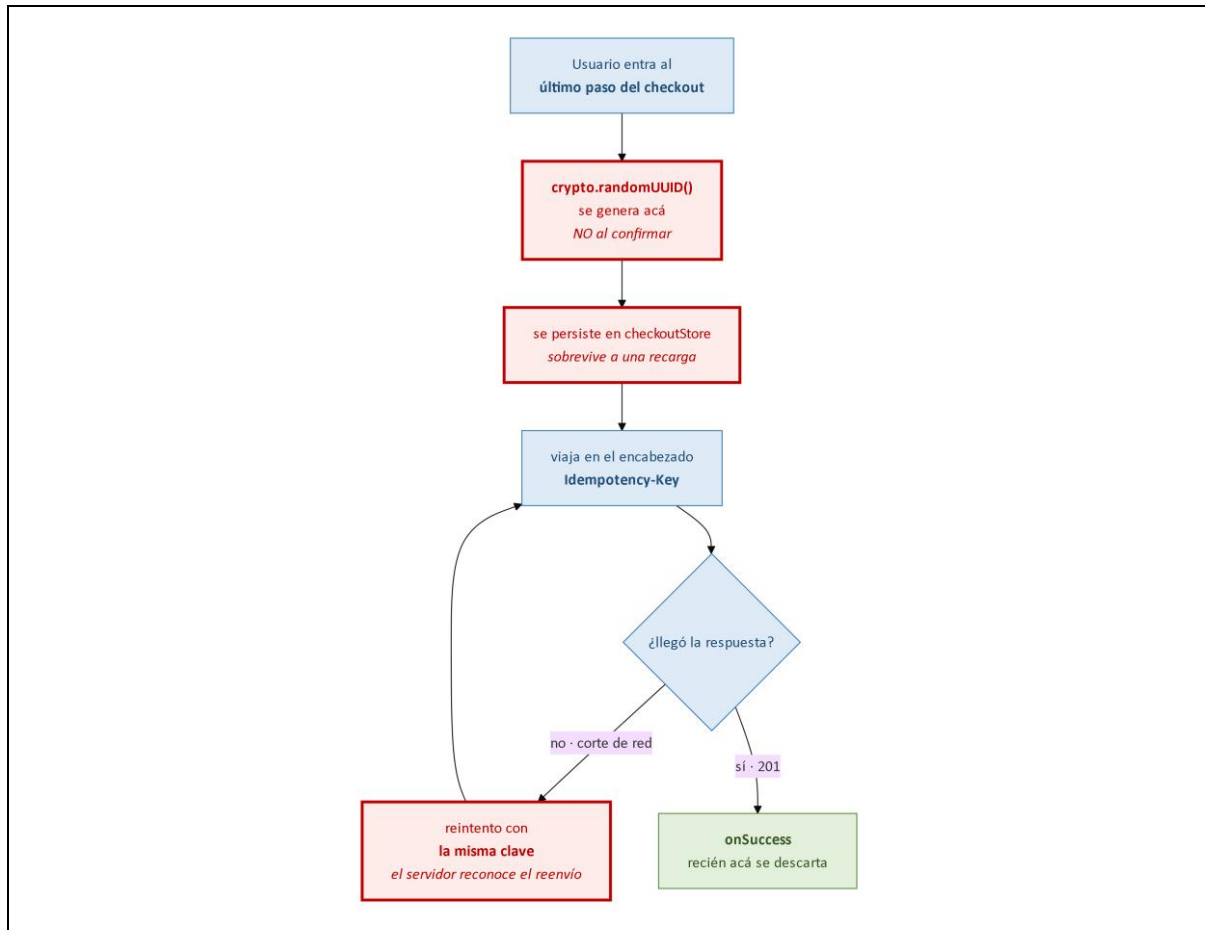


Figura 8.5. El ciclo de vida de la clave de idempotencia

OJO ACÁ: los tres bugs que terminan en un reclamo

- **Generar la clave al confirmar** → doble clic, dos pedidos, doble cobro.
- **No persistirla** → se corta el wifi, el usuario recarga y reintenta, dos pedidos.
- **Descartarla al emitir** → la respuesta tarda, el usuario reintenta, y como ya no hay clave, dos pedidos.

Los tres terminan igual: **alguien recibe dos veces la misma comida y ve dos cargos en su cuenta.**

Y lo que más me interesa que veas: **un agente de IA te va a escribir la versión con los tres bugs**, porque la versión ingenua es la que aparece en el noventa por ciento de los ejemplos de checkout de internet.

Vas a tener que hacerte las tres preguntas: *¿cuándo se genera? ¿sobrevive a una recarga? ¿cuándo se borra?* **Son el módulo entero aplicado a diez líneas.**

8.9 — Las once reglas, vistas como un sistema

Hasta acá las conociste de a una. El catálogo:

Regla	Qué exige, en una línea	Capítulo
RN-F01	Toda suscripción guarda su baja y la ejecuta al desmontar, dentro de <code>Disposable</code>	4 y 7
RN-F02	El dato externo entra por <code>textContent</code> o <code>createElement; innerHTML</code> sólo con <code>DOMPurify</code>	4
RN-F03	Servidor en el <code>QueryClient</code> , cliente en los stores; excepción, la exhibición de <code>CartItem</code>	8
RN-F04	Las guardas de ruta son usabilidad, no seguridad: el backend revalida	1 y 8
RN-F05	Una instancia de <code>Chart.js</code> por montaje, destruida al desmontar; se actualiza mutando <code>chart.data</code>	7
RN-F06	Ninguna vista se monta antes de terminar la rehidratación: carga, nunca el login	8
RN-F07	Clave al entrar al último paso, persistida, en <code>Idempotency-Key</code> , descartada en el <code>onSuccess</code>	1 y 8
RN-F08	Los importes llegan como <code>string</code> y se convierten en <code>api/</code> , nunca en la vista	3 y 6
RN-F09	Un evento invalida la clave; nunca escribe en la caché	5
RN-F10	Una sola conexión de eventos por sesión: se abre en el arranque, se cierra en el <code>logout</code>	5
RN-F11	Toda vista con datos vivos declara su intervalo de recarga de respaldo	5

No son once recomendaciones sueltas: son **cuatro problemas con sus respuestas**, y así se recuerdan mejor:

Problema de fondo	Reglas	Capítulo donde se demostró
Lo que se monta hay que desmontarlo	RN-F01, RN-F05	4 y 7
Todo dato externo es hostil hasta prueba en contrario	RN-F02, RN-F04, RN-F08	4, 5 y 6

Problema de fondo	Reglas	Capítulo donde se demostró
La red no garantiza nada	RN-F07, RN-F09, RN-F10, RN-F11	1 y 5
Cada dato tiene un dueño y un lugar	RN-F03, RN-F06	8

Y hay un patrón que atraviesa las once y vale más que la lista: **cada regla declara su garante**. Algunas nombran un test —TST-45, TST-23, TST-06—; otras dicen «revisión, sin garante automatizado». La que declara no tenerlo **le dice al equipo dónde poner los ojos**; la que finge tenerlo produce falsa confianza, peor que ninguna.

NOTA: decir qué no está verificado también es ingeniería

Cuando una regla **no** tiene garante automático, el TPI lo escribe:

Declararlo deja al equipo sabiendo que RN-F03 depende de que alguien mire; **no decir nada** hace que todos supongan que está cubierto y que el día que se rompe nadie entienda cómo pasó. Y te sirve para la entrega: si algo no llegaste a testearlo, **decilo**.

Y tres resuelven el olvido de la misma manera, que es la lección de arquitectura del módulo: **RN-F01** no dice «acordate de dar de baja», exige una clase base que acumule las bajas; **RN-F08** no dice «cuidado con el dinero», declara un tipo que impide la operación; **RN-F06** no dice «esperá la sesión», ordena una secuencia donde montar antes es imposible.

PARA ENTENDER: la única idea que sobrevive a los ocho capítulos

Las reglas que dependen de que alguien se acuerde, fallan. Las que hacen que equivocarse sea difícil, no.

No es rigor por el rigor mismo: **a las dos de la mañana antes de entregar, nadie se acuerda de nada**. Cuando diseñes algo, preguntate: *¿estoy confiando en la memoria de alguien, o estoy haciendo que el camino fácil sea el correcto?*

8.10 — Trabajar con un agente de IA sobre esta base

Acá cierra el módulo, y conviene empezar por la sección 2.4 del TPI: la modalidad **asume** el trabajo con un agente, y lo regula de dos maneras.

El repositorio debe reflejar el proceso: un historial de commits, **no un único commit final**, que no distingue un trabajo comprendido de uno pegado.

Y si el plazo no alcanza, indica el camino honesto: priorizar el núcleo —arquitectura, modelo de datos, autenticación, API, stock y su concurrencia— y dejar Redis, la cola de tareas y el tiempo real **declaradas y con su diseño entendido**. Eso dice mucho sobre qué se evalúa: **entender y declarar vale más que construir sin entender**.

El criterio que las une es el mismo de la sección 8.1, el que este módulo sostiene desde el Capítulo 1: **ninguna decisión puede copiarse sin entenderse**, porque cada una viene con su porqué y su alternativa descartada.

¿Y qué significa eso sentado frente a un agente?

Un agente propone lo más frecuente, no lo más correcto. En la masa de código que aprendió, lo mayoritario pesa más que lo bueno. Cuando coincide con lo correcto —y coincide muchas veces— el resultado es excelente; cuando no, es plausible, compila, anda en la demo... **y está mal**. Ésta es la lista de revisión del TPI:

Un agente escribe...	Y viola...	Capítulo
<code>contenedor.innerHTML = \...\${dato}...\`</code>	RN-F02	4
<code>addEventListener</code> sin su baja	RN-F01	4 y 7
<code>try { await fetch(...) } sin mirar ok</code>	El manejo de errores	5
<code>total: number</code> en el tipo de la respuesta	RN-F08	3 y 6
<code>await</code> encadenado de peticiones independientes	El tiempo de carga	5
Escribir en la caché con el contenido del evento	RN-F09	5
<code>new Chart(...)</code> en cada actualización	RN-F05	7
El <code>fetch</code> adentro del componente	La tabla de piezas	7 y 8
Todo el estado en un solo <code>store</code>	RN-F03	8
Clave de idempotencia al confirmar, o sin persistir	RN-F07	8

Diez patrones, y ninguno es un disparate: todos son lo que aparece en la mayoría de los ejemplos de internet, y los diez producen código que anda perfecto en una demo de quince minutos — que es lo que los vuelve peligrosos. De ahí salen tres formas de trabajar:

La forma	Por qué funciona
Pedir con el porqué, no sólo el qué	«Escribime la vista de pedidos» da el promedio de internet; «...los datos entran por propiedad, las suscripciones van en <code>Disposable</code> , los importes se muestran sin operar» da otra cosa. El contexto es la diferencia
Revisar contra las once reglas, no contra la intuición	«Se ve bien» no es un criterio: los diez patrones se ven bien . La lista sí lo es, y se recorre en dos minutos
Si no entendés lo que devolvió, no lo integres	El código que no entendés no lo podés arreglar , y el día que falle vas a depurar algo que nunca leíste

NOTA: para qué sirvieron los ocho capítulos

No aprendiste HTTP para saber HTTP, ni el modelo de caja para saber CSS, ni el bucle de eventos para pasar una entrevista.

Aprendiste todo eso para poder mirar cien líneas que te escribió un agente en tres segundos y decir: **«esto está bien, esto está mal, y esto de acá me falta»**.

Esa capacidad tiene un nombre viejo y es **criterio técnico**: es lo único de todo el proceso que no se puede delegar. El agente escribe más rápido que vos y siempre lo va a hacer, pero alguien tiene que saber **qué pedirle y cómo revisarlo**, y ése sos vos.

8.11 — Herramientas de diagnóstico: auditar la arquitectura desde afuera

Tres verificaciones del capítulo.

El grafo de dependencias. Hay herramientas que lo generan con los `import` estáticos del Capítulo 3 y **detectan violaciones de la regla de la sección 8.3.1**: importaciones hacia arriba, horizontales entre funcionalidades, ciclos. Es la única forma de sostener la regla entre cinco personas: **revisar a mano no escala**.

El estado en vivo. Un store expone su estado y sus suscriptores, y la caché sus claves con su antigüedad. Dos comprobaciones valen la pena: **contar suscriptores** al montar y desmontar tres veces, que si no vuelve a cero delata una fuga —lo que verifica TST-45, garante de RN-F01—; y **mirar las claves**, donde una que no debería estar suele ser estado del servidor mal guardado.

FIGURA 8.6

captura

Captura del panel de aplicación con el almacenamiento local a la vista durante el checkout: el token, los ítems del carrito y la clave de idempotencia. Es la evidencia del experimento de la sección 8.11

Figura 8.6. Stores y claves de caché en vivo

El almacenamiento persistido. El panel de aplicación muestra, sin abrir un archivo, lo que cada store guardó: ahí se verifica la tabla de 8.4.2.

EXPERIMENTO — hacelo hoy, sobre tu propio TPI

Abrí el panel de aplicación, andá al almacenamiento local y recorré la aplicación mirando **sólo eso**:

1. Iniciá sesión. **Debe aparecer el token, y no el usuario**; si aparece, hay estado del servidor en un store: RN-F03.
2. Agregá algo al carrito: aparecen los ítems con nombre y `precio_ref`, la excepción declarada de 8.4.3.
3. Abrí un modal y aplicá un filtro. **No debe aparecer nada nuevo**.
4. Entrá al último paso del checkout. **Debe aparecer la clave de idempotencia**.
5. Recargá con F5. **La clave sigue ahí** — eso es RN-F07 funcionando.
6. Confirmá el pedido: la clave desaparece recién ahora.

Seis pasos, y verificaste RN-F03, RN-F06 y RN-F07 **sin abrir un archivo de código**. Eso da una arquitectura declarada: **se puede auditar desde afuera**.

8.12 — Seguridad y evolución: qué de todo esto va a seguir sirviendo

Tres consideraciones cierran el módulo.

La arquitectura no es una defensa. Ninguna capa del frontend protege nada: RN-F04 lo dice y sus tres tests lo demuestran salteándose la interfaz. Lo que sí hace es **concentrar las decisiones**

sensibles en pocos lugares —el token en un store, la conversión de importes en la capa de acceso, la sanitización en un punto—: **pocos lugares se pueden revisar; muchos, no.**

Lo persistido sobrevive al código. Lo guardado con una versión anterior sigue ahí cuando el código cambió, y se lee con la forma nueva sin que nada avise —el caso de la sección 6.10.1—. Por eso lo persistido conviene mantenerlo mínimo: lo que declara la tabla de la sección 8.4.2.

El estado compartido es el más difícil de depurar: cuantas más piezas escriben en el mismo lugar, menos sabés quién lo cambió. **RN-F03 y la regla de dependencias acotan quién puede escribir qué:** una por dato, la otra por dirección.

Y una observación sobre la evolución: Feature-Sliced Design no es una tecnología, **es un acuerdo sobre dónde va cada cosa.** Las herramientas van a cambiar —Vite va a ser reemplazada, Tailwind va a tener competencia— y el acuerdo va a seguir sirviendo, porque los cuatro problemas de la sección 8.2 no dependen de ninguna. Lo mismo vale para los ocho capítulos: la semántica de HTTP no cambió en treinta años, el modelo de caja tiene veinticinco y el bucle de eventos es el mismo desde 1995. **Lo que cambia rápido son las herramientas; lo que se aprende acá dura.**

8.13 — Verificación: el checklist honesto

Nueve comprobaciones. **No son ejercicios: son el criterio para saber si se entendió el capítulo.**

- Ubicar diez archivos del frontend del TPI en su capa y **justificar qué puede importar cada uno.** (8.3.2)
- Encontrar una importación que viole la regla y explicar **cuál de las dos prohibiciones rompe.** (8.3.1)
- Clasificar diez datos como estado del cliente o del servidor, justificando **por su dueño.** (8.4.1)
- Explicar por qué `authStore` persiste el token **y no el usuario.** (8.4.2)
- Explicar por qué `eventosStore` persiste el último identificador, con la sección 5.9.3. (8.4.2)
- **Reproducir el parpadeo** del login quitando el estado de rehidratación, y corregirlo con la unión discriminada. (8.6)
- Verificar en el panel de aplicación que la clave de idempotencia **aparece al entrar al último paso, sobrevive a la recarga y desaparece al confirmar.** (8.8 y 8.11)
- Explicar por qué los garantes de RN-F04 **no usan el frontend.** (8.7)
- Revisar código generado por un agente **contra los diez patrones**, documentando cuáles aparecieron. (8.10)

8.14 — Los doce errores frecuentes

Todos tienen algo en común: **en el momento, no parecen errores.** Por eso son frecuentes.

El error	Por qué duele	Sección
Organizar por tipo de archivo	Reparte cada cambio entre cinco carpetas y hace imposible borrar	8.2
Importar hacia arriba	El store pierde independencia y arrastra medio sistema	8.3.1
Importar en horizontal entre funcionalidades	Rompe la unidad de la carpeta; lo compartido baja de capa	8.3.1
Poner el estado del servidor en un store	Te hace reimplementar caché e invalidación. Viola RN-F03	8.4.1
Persistir el usuario junto con el token	Sus datos pueden haber cambiado, empezando por el rol	8.4.2

El error	Por qué duele	Sección
Confundir «todavía no sé» con «no»	El parpadeo del login en cada recarga. Viola RN-F06	8.6
Creer que las guardas protegen algo	Son usabilidad; el cartel no es cerradura. Es RN-F04	8.7
Generar la clave de idempotencia al confirmar	Un doble clic: dos pedidos y un doble cargo	8.8
No persistir la clave	Una recarga tras un corte de red: dos pedidos	8.8
Descartar la clave al emitir la petición	Un reintento antes de la respuesta: dos pedidos	8.8
Llamar a la API desde un componente	Lo ata a una ruta; el componente recibe datos	8.3.2 y 7.6.4
Integrar código de un agente que no se entiende	El día que falle vas a depurar algo que nunca leíste	8.10

8.15 — Las actividades, y qué busca cada una

Siete actividades, con lo que cada una busca.

1. Mapa del proyecto

Dibujar el árbol de carpetas del frontend del TPI siguiendo la tabla de la sección 8.3.2, con al menos tres funcionalidades, y escribir para cada capa qué puede importar.

Qué busca: *que la estructura deje de ser una convención copiada y pase a ser una decisión que podés justificar.*

2. Auditoría de dependencias

Sobre un proyecto propio, generar el grafo de importaciones y documentar las violaciones de la regla de la sección 8.3.1, proponiendo para cada una si corresponde **bajar algo de capa** o **invertir la dependencia**.

Qué busca: *que veas el desorden medido y no opinado. Un grafo no discute.*

3. La distinción, aplicada

Listar quince datos del TPI y clasificarlos como estado del cliente o del servidor, **justificando por su dueño**, e identificar los casos dudosos.

Qué busca: *los casos dudosos: ahí la regla se entiende de verdad, y `precio_ref` es el primero que aparece.*

4. El arranque completo

Implementar los siete pasos de la sección 8.6 con la unión discriminada de tres estados, demostrando que la vista de login **no puede montarse** durante la rehidratación y por qué el compilador ayuda.

Qué busca: *la diferencia entre acordarse de algo y volverlo imposible de olvidar.*

5. El ciclo de la clave

Implementar RN-F07 completa y verificar los tres casos de fallo de la sección 8.8, documentando qué pasa **con la implementación correcta y con las tres versiones defectuosas**.

Qué busca: *que veas los tres bugs pasar delante tuyo. Después no se olvidan más.*

6. Exploración: revisión de código generado

Pedirle a un agente una vista completa del TPI **sin instrucciones sobre las reglas**, revisarla contra los diez patrones de la sección 8.10 y documentar cuáles aparecieron. Repetir dando el contexto de las reglas, **midiendo cuántos patrones desaparecieron**.

Qué busca: *que midas con un número la diferencia que hace el contexto.*

7. Exploración: la arquitectura auditada desde afuera

Sobre la aplicación en ejecución, verificar RN-F03, RN-F06 y RN-F07 **usando únicamente el panel de aplicación y el de red**, sin abrir el código, documentando qué evidencia sostiene cada regla y relacionándolo con la sección 8.11.

Qué busca: *que descubras que una arquitectura bien declarada se verifica sin leer el código. Eso es lo que hace un revisor.*

8.16 — Síntesis: las doce frases

1. Organizar por tipo de archivo colapsa por cuatro razones, y la más cara es la menos visible: **donde no se puede borrar, un proyecto sólo puede crecer**.
2. Feature-Sliced Design adapta al frontend dos ideas del backend: **organizar por dominio y declarar una dirección para las dependencias**. No es una tecnología: es un acuerdo.
3. Las dos prohibiciones resuelven cosas distintas: **no importar hacia arriba** hace independientes las capas de abajo; **no importar en horizontal** mantiene cada funcionalidad como una unidad borrrable.
4. La pieza transversal del canal de eventos **está declarada, nombrada y acotada**. Una regla con excepciones declaradas es una regla; con excepciones silenciosas es una sugerencia.
5. **Cada dato tiene un dueño**. Del servidor, es una copia que puede quedar vieja y va en la caché; si sólo existe en el navegador, va en un store. Confundirlos es reimplementar a mano una caché, mal.
6. `authStore` persiste el token **y no el usuario**, porque un usuario persistido sería estado del servidor en un store. `eventosStore` persiste el último identificador para que **una recarga no pierda el hueco de eventos**.
7. **«Todavía no sé» es un estado y hay que representarlo**. No hacerlo produce el parpadeo del login, y una unión discriminada lo vuelve imposible de escribir.
8. El garante de RN-F04 **no usa el frontend**: ejercita el backend directamente, que es lo que haría un atacante. Convierte una advertencia en una verificación.
9. Los tres detalles de RN-F07 —cuándo se genera la clave, que se persista, cuándo se descarta— son **tres bugs distintos**, y los tres terminan en un doble cargo.
10. Las once reglas son **cuatro problemas con sus respuestas**, y cada una declara su garante, incluso cuando es «revisión, sin automatizar». Eso le dice al equipo dónde poner los ojos.
11. **Las reglas que dependen de que alguien se acuerde, fallan. Las que hacen que equivocarse sea difícil, no**. Es la lección de arquitectura del módulo entero.
12. Un agente propone **lo más frecuente, no lo más correcto**, y el módulo mostró los diez lugares donde eso no coincide. Saber cuáles son es lo que ningún agente reemplaza.

8.17 — Qué leer, y en qué orden

Las fuentes de este capítulo no son especificaciones sino literatura de arquitectura: el orden importa más que nunca.

Si lees una sola cosa

Robert C. Martin, *Clean Architecture* (Prentice Hall, 2017), y dentro del libro **el capítulo sobre la regla de dependencia** —apuntan siempre hacia adentro—, fundamento directo de la sección 8.3.1. Su noción de *arquitectura que grita* es la segunda crítica de la sección 8.2, y el libro desarrolla su artículo de 2012.

Si lees tres

- **Alistair Cockburn**, *Hexagonal Architecture* (2005), o *puertos y adaptadores*: el artículo original sigue en alistair.cockburn.us y es corto. Es la formulación más temprana de lo que la sección 8.2 resume en una línea.
- **Feature-Sliced Design**, la documentación oficial en feature-sliced.design, con su vocabulario de capas y segmentos. Conviene leerla **sabiendo que el TPI la adapta**: toma su principio de dependencias y su organización, con nomenclatura propia.
- **TanStack Query y Zustand**. La primera discute la distinción entre estado del cliente y del servidor —la sección 8.4.1— mejor que la mayoría de los textos de arquitectura, **porque parte del caso concreto**. La segunda cubre la composición de middlewares idéntica en los seis stores, con la persistencia parcial —el `partialize` de 8.4.2— y la suscripción por selector.

Las secciones del TPI que conviene tener abiertas

- **2.4** — la regla de dependencias y la tabla de piezas (sección 8.3).
- **2.5** — las once reglas con sus garantes (sección 8.9).
- **13.1** — los seis stores y qué persiste cada uno.
- **13.3** — los siete pasos del arranque.
- **2.4 de la Primera Parte** — el trabajo con agentes, que cierra la sección 8.10.

Y como cierre del módulo, tres lecturas que exceden lo técnico

- **Hunt y Thomas**, *The Pragmatic Programmer* (2.ª edición, Addison-Wesley, 2019): su primer capítulo trata la **responsabilidad sobre el propio trabajo**, que es lo que discute la sección 8.10 sobre integrar código que no se entiende.
- **Brooks**, *No Silver Bullet* (1986): por qué ninguna herramienta elimina la complejidad **esencial** de un problema, sólo la accidental. Se lee muy distinto hoy que hace cuarenta años.
- **Petzold**, *Code* (2.ª edición, Microsoft Press, 2022): del camino que va de una lamparita a una computadora. Es la mejor respuesta a la pregunta que este módulo intentó contestar ocho veces — **por qué conviene entender lo que hay debajo de la herramienta que uno usa**.

Cierre: las ocho cosas que hay que recordar

Ocho capítulos, ocho problemas, ocho conjuntos de decisiones que alguien tomó y que hoy explican por qué la web es como es. Si dentro de un año te acordás de ocho frases, que sean éstas.

LAS OCHO

1. **La web resignó garantías para no necesitar coordinación.** La pregunta ante cualquier tecnología no es sólo si funciona: es **qué resignó para funcionar**.
2. **Ningún concepto se entiende sin su problema.** Por eso cada capítulo empezó por lo que se usaba antes y por qué colapsó.
3. **Lo que se monta hay que desmontarlo:** un `addEventListener`, una suscripción, un gráfico de `Chart.js`. El mismo lugar: `destroy()`.
4. **Todo dato externo es hostil hasta prueba en contrario**, y el dinero ni se toca en el frontend.

5. La red no garantiza nada. De ahí la clave de idempotencia, la invalidación en lugar de la escritura, y el intervalo de respaldo.

6. Cada dato tiene un dueño y un lugar. Del servidor, a la caché; del navegador, a un store. Y las excepciones se declaran y se acotan.

7. «Todavía no sé» es un estado, y el caso que casi todos olvidan es el que produce el parpadeo del login.

8. Las reglas que dependen de que alguien se acuerde, fallan. Las que hacen que equivocarse sea difícil, no.

Y una novena, que no está escrita en ningún capítulo pero está en todas sus páginas. El TPI espera un sistema funcionando, sí, pero lo que evalúa —y lo dice en su fundamentación— es **que ninguna de sus decisiones se haya copiado sin entenderse.**

Para eso fue el módulo: no para que sepas escribir un `addEventListener`, que eso lo escribe un agente en un segundo, sino **para que cuando lo escriba, del otro lado haya alguien que sepa preguntar dónde está la baja.**